Pseudocode Review, Fix-It, And Revision

Generated Jul 9, 2026 6:13 AM

What We'll Learn

By the end of this lesson, students can review pseudocode, find common mistakes, and revise pseudocode to make it clear, complete, and logically correct.

Words We'll Use

- Pseudocode
- Review
- Debug
- Revise
- Logic error
- Sequence
- Condition
- Input
- Process
- Output

Before We Start

Have you ever written an answer that looked correct at first, but later you noticed a missing step or wrong order?

In programming, reviewing our work is important because even small mistakes can change the result of a program.

Let's Understand

Pseudocode is a plain-language plan for a program. Before writing actual code, we use pseudocode to organize the steps of our solution.

However, pseudocode can still have mistakes. Some common mistakes are:

- Missing `START` or `END`
- Missing input
- Missing output
- Steps are in the wrong order
- Condition is unclear
- `IF`, `ELSE`, or `END IF` is incomplete
- Wrong use of action words such as `INPUT`, `DISPLAY`, `SET`, or `COMPUTE`
- The logic does not solve the problem

Reviewing pseudocode means checking if the steps are complete, clear, and arranged correctly.

Fixing pseudocode means correcting the mistakes.

Revising pseudocode means improving the solution so it is easier to understand and follow.

We will also use proper indentation. The main steps inside `START` and `END` are indented. Steps inside `IF` and `ELSE` are indented again.

Example format:

```
START
    INPUT value

    IF value > 0 THEN
        DISPLAY "Positive"
    ELSE
        DISPLAY "Not Positive"
    END IF
END
```

Let's Look at Examples

Example 1: Fix Missing Input

This example shows how to fix pseudocode with a missing input.

Problem:

Create pseudocode that asks for the user's name and displays a greeting.

Incorrect Pseudocode:

```
START
    DISPLAY "Hello, " + name
END
```

Problem with the pseudocode:

The variable `name` is used, but the user was never asked to enter the name.

Corrected Pseudocode:

```
START
    INPUT name
    DISPLAY "Hello, " + name
END
```

Explanation:

The corrected pseudocode first asks the user to enter a name. After that, it displays the greeting using the name entered by the user.

> **Common mistake**
>
> Students may use a variable without asking for its value first.

> **Mini-check**
>
> What line was missing in the incorrect pseudocode?

Example 2: Fix Wrong Order of Steps

This example shows how to fix pseudocode with steps in the wrong order.

Problem:

Create pseudocode that asks for two numbers, computes their sum, and displays the result.

Incorrect Pseudocode:

```
START
    COMPUTE sum = num1 + num2
    INPUT num1
    INPUT num2
    DISPLAY sum
END
```

Problem with the pseudocode:

The sum is computed before the values of `num1` and `num2` are entered.

Corrected Pseudocode:

```
START
    INPUT num1
    INPUT num2
    COMPUTE sum = num1 + num2
    DISPLAY sum
END
```

Explanation:

The program must first receive the two numbers. After the input is complete, it can compute the sum and display the result.

> **Common mistake**
>
> Students may write the process step before the input step.

> **Mini-check**
>
> Why should `INPUT num1` and `INPUT num2` come before computing the sum?

Example 3: Fix Incomplete Decision

This example shows how to fix pseudocode with an incomplete decision structure.

Problem:

Create pseudocode that asks for a grade. If the grade is greater than or equal to 75, display "Passed". Otherwise, display "Failed".

Incorrect Pseudocode:

```
START
    INPUT grade

    IF grade >= 75 THEN
        DISPLAY "Passed"
    DISPLAY "Failed"
END
```

Problem with the pseudocode:

The decision is incomplete. The pseudocode does not use `ELSE` and `END IF`, so the logic is confusing.

Corrected Pseudocode:

```
START
    INPUT grade

    IF grade >= 75 THEN
        DISPLAY "Passed"
    ELSE
        DISPLAY "Failed"
    END IF
END
```

Explanation:

The corrected pseudocode clearly shows two possible results. If the grade is 75 or higher, the program displays "Passed". Otherwise, it displays "Failed".

> **Common mistake**
>
> Students may forget to use `ELSE` and `END IF` in a decision structure.

> **Mini-check**
>
> What keyword is used for the second possible result?

Now We Apply

Create a corrected and improved pseudocode for the problem below.

Problem:

Ask for the student's name and score. If the score is greater than or equal to 75, display the student's name and "Passed". Otherwise, display the student's name and "Failed".

Your pseudocode must include:

- `START`
- Input for student name
- Input for score
- Correct `IF-ELSE` decision
- Correct output
- `END IF`
- `END`
- Proper indentation

Sample Answer:

```
START
    INPUT studentName
    INPUT score

    IF score >= 75 THEN
        DISPLAY studentName + " Passed"
    ELSE
        DISPLAY studentName + " Failed"
    END IF
END
```

Challenge Task

You are given incorrect pseudocode. Review, fix, and revise it.

Problem:

Ask for two numbers. If the first number is greater than the second number, display "First number is greater". Otherwise, display "Second number is greater or equal".

Incorrect Pseudocode:

```
START
    INPUT num1

    IF num1 > num2 THEN
        DISPLAY "First number is greater"
        DISPLAY "Second number is greater or equal"
END
```

Your task:

1. Identify all mistakes.
2. Write the corrected pseudocode.
3. Explain why your correction is better.

Expected corrected pseudocode:

```
START
    INPUT num1
    INPUT num2

    IF num1 > num2 THEN
        DISPLAY "First number is greater"
    ELSE
        DISPLAY "Second number is greater or equal"
    END IF
END
```

Possible mistakes to identify:

- `num2` is used but not entered.
- `ELSE` is missing.
- `END IF` is missing.
- The output statements are not separated into true and false branches.

How Our Work Will Be Checked

Criteria	Excellent	Good	Needs Improvement
Error Identification	All mistakes are correctly identified	Most mistakes are identified	Many mistakes are missed
Correct Input	All needed inputs are present	One input is unclear	Input is missing or incorrect
Correct Process	Process is clear and correct	Process has a minor issue	Process is missing or incorrect
Correct Decision	IF-ELSE logic is complete and correct	Decision has a minor issue	Decision is missing or incorrect
Correct Output	Output is complete and clear	Output is partly clear	Output is missing or confusing
Sequence and Order	Steps are arranged correctly from START to END	One step is slightly misplaced	Steps are confusing or incorrect
Indentation	Pseudocode uses clear and consistent indentation	Indentation has minor errors	Indentation is missing or confusing
Revision Quality	Pseudocode is improved and easy to understand	Pseudocode is mostly clear	Pseudocode is still unclear
Explanation	Student explains the correction clearly	Explanation is partly clear	Student cannot explain the correction

Let's Reflect

Answer the following questions:

1. What mistake do you commonly see in pseudocode?
2. Why is it important to review pseudocode before writing code?
3. Which is harder: finding the mistake or fixing the mistake? Why?
4. How does indentation help make pseudocode easier to read?
5. How can pseudocode help you avoid errors in programming?

How We Show Learning

Students will show learning by:

- Finding errors in incorrect pseudocode
- Correcting missing or misplaced steps
- Revising unclear pseudocode
- Explaining the corrected logic
- Creating a complete pseudocode solution using input, process, decision, and output
- Using proper indentation in pseudocode

Resources / References

- Java toolchain guide
- Pseudocode review checklist
- Previous lesson examples